

Basic to Intermediate Interview Questions

1. HTML

1. What is `<!DOCTYPE html>`?
 - is a declaration that specifies the document type and version of HTML used in a web page.
2. What is semantic HTML?
 - HTML 5 introduced HTML tags such as header, section, nav, footer, articles, sections. Which helps to understand the search engine and the site's content properly.
3. What are HTML entities?
 - Special code to render special characters to avoid conflicts. E.g `&` for `&`, `<` for `<`
4. What is the purpose of head & body tag?
 - a. Head tag is for meta data
 - b. Body tag for actual representation of ui
5. What is the difference between an inline and block-level element?
 - Inline: Align the container/blocks into a single line
 - Block: Align the containers in columns, one after the other.
6. What is the `<a>` tag used for?
 - Use for hyperlink content.
7. What is the difference between the `<div>` and `<section>` tags?
 - `<div>` is a generic container.
 - `<section>` is thematic grouping. Grouping the similar contents.
8. What are HTML5 input types?
 - `type="email"`, `type="url"`, `type="tel"`, `type="date"`, `type="number"`, `type="range"`
9. What is the difference between `localStorage` and `sessionStorage`?
 - `localStorage` stores data with no expiration time, so it persists even after the browser is closed.

- `sessionStorage` stores data for the duration of the page session, and it is cleared when the page session ends (i.e., when the browser or tab is closed).

10. Explain the use of the `defer` and `async` attributes in the `<script>` tag.

- `async` loads the script asynchronously, meaning it doesn't block the rendering of the page.
- `defer` ensures that the script is executed after the HTML is fully parsed but before `DOMContentLoaded`.

11. What is the `viewport` meta tag and why is it important for responsive design?

- The `viewport` meta tag controls the layout of the page on mobile devices, defining the visible area of the page. It is crucial for making web pages responsive, especially on mobile devices.

12. What are the differences between the `<iframe>` and `<object>` elements?

- `<iframe>` is used to embed another HTML page or external content into a page.
- `<object>` can embed a variety of content types, such as Flash, PDFs, and other resources, but it's less commonly used for embedding entire pages.

13. What is the `<picture>` element used for?

- Used to provide multiple image sources for different scenarios. Such as for different screens, types, themes, devices etc. E.g for different screen sizes.

```
<picture>
  <source media="(max-width: 600px)" srcset="small-image.jpg">
  <source media="(max-width: 1200px)" srcset="medium-image.jpg">
  
</picture>
```

14. What are the `application/x-www-form-urlencoded` and `multipart/form-data` content types?

- `application/x-www-form-urlencoded`: for key value pairs data
- `multipart/form-data` : for multimedia type data.

15. Explain the shadow DOM in relation to HTML elements.

- The Shadow DOM is a web platform feature that allows for the encapsulation of a part of the DOM inside an element, enabling isolated

styles and behavior, which prevents them from interfering with the global document.

16. Key SEO Tag's

- **og from facebook**

```
<meta property="og:title" content="Best Web Development Tutorials">
<meta property="og:description" content="Comprehensive tutorials and guides to help you master web
development.">
<meta property="og:image" content="https://www.example.com/thumbnail.jpg">
```

- **twitter from twitter**

```
<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:title" content="Best Web Development Tutorials">
<meta name="twitter:description" content="Comprehensive tutorials and guides to help you master web
development.">
<meta name="twitter:image" content="https://www.example.com/thumbnail.jpg">
<meta name="twitter:creator" content="@username">
<meta name="twitter:site" content="@sitehandle">
```

- **Google**

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "Learn Web Development with This Guide",
  "author": "John Doe",
  "datePublished": "2024-12-01",
  "image": "https://www.example.com/image.jpg"
}
</script>
```

17. In html, the id attribute must be unique.

2. CSS

1. What are the different types of CSS selectors?

- **Element selector:** Selects all elements of a specific type (e.g., `p {}`).
- **Class selector:** Selects all elements with a specific class (e.g., `.example {}`).
- **ID selector:** Selects a specific element with an ID (e.g., `#header {}`).
- **Universal selector:** Selects all elements (`* {}`).
- **Attribute selector:** Selects elements based on attributes (e.g., `input[type="text"] {}`).

2. What is the difference between `class` and `id` in CSS?

Class: Can be used multiple times in a document. It is prefixed with a dot (`.`) in CSS.

ID: Must be unique within a page. It is prefixed with a hash (`#`) in CSS.

3. What are pseudo-classes in CSS?

Used to define the special state of an element.

`:hover`: Applied when the user hovers over an element.

`:focus`: Applied when an element has focus (e.g., form input).

`:nth-child(n)`: Applied to elements based on their position in a group of siblings.

`:active`: Applied when an element is being activated (e.g., a button click).

4. Position in css (static, absolute, relative, fixed, sticky)

Static: is default position for all elements

Absolute: is positioned based on its closest ancestor if and only if the ancestor's position is defined.

Relative: is positioned based on its current position in the dom.

Fixed: will be fixed in the given position

Sticky: will be fixed in the position it mentioned while scrolling, else it behaves normal like Elements.

5. What are the advantages of using `rem` and `em` units in CSS?

rem (root em): Relative to the root element's font size (usually `html`).

em: Relative to the font size of the current element. These units help in creating more scalable and accessible designs, especially for responsive web design.

6. What is the difference between `display: none` and `visibility: hidden`?

display: none: The element is removed from the document layout and does not occupy space.

visibility: hidden: The element is still present in the layout and occupies space but is not visible to the user.

7. Explain the **flex-shrink** and **flex-grow** properties in Flexbox.

flex-shrink: Defines the ability of a flex item to shrink if necessary. The default value is **1**, meaning the item will shrink if the container is too small.

flex-grow: Defines the ability of a flex item to grow if there is available space. The default value is **0**, meaning the item will not grow unless specified.

8. What are CSS variables (Custom Properties)?

It allows you to define variables in css for reusing those. Need to define `--` before defining the variable.

```
:root {  
  --main-color: #3498db;  
}  
.box {  
  background-color: var(--main-color);  
}
```

9. What are **::before** and **::after** pseudo-elements in CSS?

::before and **::after** are pseudo-elements used to insert content before or after an element's actual content. These are often used for adding decorative elements or icons.

3. React Questions:

1. What are some common anti-patterns in React development, and how can they be avoided?

- a. Mutating the state directly.
- b. Forgetting to clean up time intervals.
- c. Using deprecated lifecycle methods/hook
- d. Not using key in list rendering.
- e. Excess use of useCallback, useMemo, useEffect.
- f. Extreme nested props drilling.
- g. Using class component

2. Explain the differences between controlled and uncontrolled components in React. When would you use each type?

- a. Controlled:
 - i. controlled by the developer.
 - ii. Need state to store values.
 - iii. When needs custom validation, use it
- b. Uncontrolled:
 - i. Controlled by DOM.
 - ii. Need ref to get values.
 - iii. When no needs of validation, use it

3. Explain the concept of "lifting state up" in React. When and why would you do it?

- **Lifting state up** in React is the design pattern of moving shared state to the nearest common ancestor component so that multiple child components can access and synchronize it via props.

4. Discuss the React reconciliation algorithm. What is it, and how does it optimize Rendering?

1. Store the light weight virtual dom in memory representation.
2. Create another virtual dom & compare it with the in memory represented virtual dom using the diffing algorithm.
3. If it finds something different, it updates the real dom.

It uses Fiber architecture which breaks the rendering process into chunks, & allow react to pause, prioritize, abort the rendering process. So that it works faster in an optimized way.

5. What is the role of Webpack or other bundlers in a React application? Discuss how tree-shaking and code-splitting can optimize your bundle size ?

- Bundler: Is a tool that packs/bundles all the files, dependencies, assets in a format that can be served to the browser efficiently.
- Tree Shaking: a technique that removes the unused code from the bundle.
- Code Splitting: a technique that split the code into smaller chunks so that browser can load the code which is required for the current ui.

6. Discuss how to implement optimistic UI updates in a React application. What challenges might arise, and how can they be managed?

- Optimistic UI is a process where the user interface is immediately updated based on the assumption that an API call will succeed. After the API call succeeds, the UI is synchronized with the server response. If the API call fails, the UI is reverted or updated to reflect the error.
- We can use the 'useOptimistic()' hook to implement the optimistic UI in react. It takes an initial value & a reducer function as parameters & return the current (optimistic) state & a reducer function

7. How does the Context API handle performance concerns with large component trees? What strategies can be employed to avoid unnecessary Re-renders?

- It can pass data through the component tree in react without props drilling. If it isn't used properly, it may be a performance issue.
- To avoid unnecessary re-renders, we can use a few techniques.
 - a. Splitting the context in different parts.
 - b. Can memoize the context value.

8. Discuss the differences between shallow comparison and deep comparison in React? How does React use shallow comparison for props and state updates?

- Shallow comparison: compares 2 object's references
- Deep comparison: compare 2 objects's content nestedly.

9. What is the difference between props and state?

- **Props** are read only data, passed through the components.
- **States** are immutable data used as components of their own property. Need to update through the setState function

10. What is the component?

- Basic block of reusable elements in react.

11. What is conditional rendering: why is it used?

- Rendering the components based on certain conditions. We used ternary or boolean operators mostly for conditional rendering

12. Purpose of useEffect in react ?

- Is a hook that serves all the lifecycles of a react component.

13. What is react useContext ?

- A way to handle state globally in react.

14. What is the best way to compute large calculations in react.

- Using useCallback() hook.

15. How do you optimize performance in a React application?

- using lazy loading,
- using React Memo,
- using useCallback,
- using useMemo

16. What is the output of the following component?

```
import { useEffect, useState } from 'react'
const Test = () => {
  console.log('Hello World 1')
  const [state, setState] = useState(() => {
    console.log('Hello World 3')
    return 0
  })

  useEffect(() => {
    console.log('Hello World 2')
  }, [])

  useEffect(() => {
    console.log('Hello World 4', state)
  }, [state])

  return <>{console.log('Hello World')}</>
}

export default Test
```


17. How could we render a large list in react?

- By using React Virtualization

18. React Life cycle?

- Mounting
- Updating
- Unmounting

19. Is index good/bad to use as key in list rendering? Why with an example?

- It's bad to use an index as the key of a list if the list is dynamic. When we push/pop the elements . react will behave inconsistent .

20. Unmount in Functional component?

- We can add return in useEffect hook's callback function.

21. Updating state isn't async nor sync, how could I get the updated value of state in the exact next line in the same function call?

- In this scenario , we can useCallback function inside the setState() function.

22. Flux Architecture in React ?

- is a design pattern for managing **unidirectional data flow** in applications, often used with React. It divides the application into four main components: **Actions**, **Dispatcher**, **Store**, and **View** .

23. Why do we need key while we render Component inside .map(())=>(<div key></div>)?

- because it helps React efficiently identify and manage elements during the **reconciliation process**

24. What is PureComponent in react?

- **PureComponent** is a special type of React component that performs a shallow comparison of props and state to determine whether the component needs to re-render.

25. What is Portal in react?

- A **React Portal** provides a way to render children components outside the parent component's DOM hierarchy.

26. Composition & Inheritance in React?

- Inheritance made the component tight coupled. That is why react uses composition over inheritance.

27. Handling errors in React?

- Using trycatch for asynchronous operations.
- Using optional chaining
- Using Error boundary with fallback component

28. What is Strict Mode in React ?

- **React's Strict Mode** is a development tool that helps identify potential problems in an application. It does not render anything visible in the UI and does not affect the production build. Instead, it activates additional checks and warnings for its descendants during development to encourage writing safe and efficient code.

29. ref/forward ref?

- A **ref** is a reference to a DOM element or React component.
- **forwardRef** is a higher-order function that allows a parent component to pass down a **ref** to a child component.

N.B: from react 19, we can directly pass ref as props.

30. Design Pattern in React, (Render Props Pattern, HOC , State Lifting,)

- **HOC**: is a function that takes a component and returns a new component, often with additional props, logic, or functionality. HOCs are used to share functionality across multiple components in a reusable way.

31. useState, useTransition, useOptimistic, useMemo, useCallback

- **useState**: `useState` is a Hook that allows you to update state based on the result of a form action. **React 19**
- **useTransition**: `useTransition` is useful when you have non-urgent state updates that don't need to block the user interface, such as when filtering a list, searching, or lazy loading data
-

32. How could we stop the child component's re-rendering if the parent component's state gets updated?

- We have to make the child component as a pure component. To make the component pure, we can use `memo()` function of react or `PureComponent` in class component

33. What can happen if we use `useMemo` incorrectly?.

- It will cause memory leakage.

34. React's Concurrent Mode?

30. How can you share logic between components without repeating code?.

31: `useLayoutEffect`, `useTransition`, `useDeferredValue`?

32. React Fiber?

33. Code splitting in react?

34. WebAssembly integration with React?

35. Web Worker integration with React ?

36. Render Props Pattern ?

37. Do React native use the same virtual dom in its inside?

23. What are the implications of using mutable state with hooks? How do you ensure that your components remain predictable and avoid side effects?

24. How does React's Concurrent Mode improve the user experience? Discuss its features and how to implement it in a project.

11. How does the React Fiber architecture improve reconciliation performance? Discuss the concept of "time slicing" and "prioritization."

12. Explain the concept of "context selectors" in React. How can you optimize performance when using the Context API to avoid unnecessary re-renders?

13. What are the implications of using `useImperativeHandle` with `forwardRef`? When would you choose to use them together, and what are the potential Drawbacks?

14. Explain the differences between React's `SyntheticEvent` and native browser events. How does React handle event delegation, and what are the benefits?

15. Discuss the concept of "composition" in React. How does it differ from inheritance, and what are the advantages of using composition in component Design?

16) Discuss how to implement a custom notification system in a React application. What design patterns would you use to ensure flexibility and ease of use?

4. Javascript

1. Introduction to JavaScript
2. Basic Syntax and Types
3. Control Structures
4. Functions
5. Objects and Arrays
6. Asynchronous JavaScript
7. The DOM (Document Object Model)
8. Modules and Bundling
9. Web APIs and Fetch
10. Closures
11. Prototypal Inheritance
12. the “this” keyword
13. Design Patterns
14. Performance Optimization (de bouncing, throttling, minimizing DOM manipulations).
15. *Modern JavaScript Features (Template Literals, Spread and Rest Operators, Async Iterators)*
16. TypeScript
17. Promises and Async/Await
18. *Event Loop and Concurrency Model*
19. Higher-Order Functions
 - A function that takes another function as an argument or returns a function.
20. Functional Programming Concepts
21. Design Patterns
22. JavaScript Engine Internals
23. Regular Expressions
24. Memory Management and Performance Optimization
25. Service Workers
26. Dynamic Programming with JavaScript
28. What are the differences between shallow copy and deep copy in JavaScript?
How can you implement each?
 - shallow copy: copy the reference of data.
 - Deep copy: copy the actual value of data. Even nested.

29. What is the event loop in JavaScript? How does it handle asynchronous code Execution?

- Event loop is a component of Javascript engine. It sits between the task queue & callstack. Keeps checking the task queue and if found any callback, it placed that into the callstack. It manages the execution of asynchronous code in javascript.

30. What are IIFEs (Immediately Invoked Function Expressions), and why are they used? Provide an example.

- An IIFE is typically a **function expression** that is invoked right away by adding parentheses around the function definition and another set of parentheses to call it immediately
- Avoid global scope pollution
- Used in modular pattern

31. What is the difference between `Object.freeze()` and `Object.seal()`? When Would you use each?

- `Object.freeze()` : make the object completely immutable.
- `Object.seal()`: prevent adding new properties. Still can modify the values of existing properties.

32. Discuss the differences between synchronous and asynchronous programming. How does JavaScript handle asynchronous operations?

33. What are promises in JavaScript? Explain the difference between `Promise.all()`, `Promise.race()` and `Promise.any()`.

- **Promise:** A javascript object that helps to work with asynchronous programming. It has 2 params resolve, reject. If an asynchronous operation fails, we can reject it, else we can resolve it.
- Promise has 3 state, Pending, fulfilled, rejected
- **Promise.all():** takes an array of promises, returns a single result if all the promises get fulfilled else even a single promise fails, it returns rejected.
- **Promise.race():** resolves with the first promise that completes (either resolves or rejects)
- **Promise.any():** resolves any fulfilled.

34. What is this? What is the purpose of `bind()`, `call()`, and `apply()` methods? How do they differ from each other?

- The **this** keyword refers to the context where a piece of code, such as a function's body, is supposed to run
- `call()`: we explicitly say who is the value of this.
`printName.call(jahid)`

Here jahid inside the printName function refers to this

- apply(): similar like call(), just passes arguments as array.
- bind(): similar like call(), it returns a function instead of calling.

35. Explain the concept of "memoization" and how it can improve performance in Functions.

- Memoization is an optimization technique in programming where you **cache** the results of expensive function calls and reuse those cached results when the same inputs occur again

<https://gist.github.com/teddy-teem/dced2fd63004a617c6ebbf4f52e878a9>

36. What is the `new` keyword in JavaScript? How does it work, and what happens under the hood when you create a new instance of an object?

- Create a new {} empty object.
- Set the prototype of the constructor function.
- Call the constructor function.
- Return the object

37. Explain the concept of "event delegation" in JavaScript. Why is it useful in managing events?

- Let's say I have an ordered list:

```
<ol>
  <li>A</li>
  <li>B</li>
</ol>
```

If we add a single **click** event listener to the `<ol>` element, the event will "bubble up" from any `<li>` inside the `<ol>` when clicked. This technique, called **event delegation**, allows us to manage events for all the list items using just one event listener on the parent element.

This is not simply "called event listener" — the correct term for this approach is **event delegation**.

38. What is the difference between an "object" and a "map" in JavaScript? When would you choose one over the other?

- Key type: objects use string & symbol, where map use any type such as object, function number etc.
- Orders: Object doesn't guarantee order, map guarantee insertion order.

- Object inherits Object.prototype property where map doesn't inherit it.
- Object is general purpose structure data where map is frequently updated key value pairs

39. What are `Symbol` and `BigInt` in JavaScript? How do they differ from traditional data types?

- Symbol: A **Symbol** is a unique and immutable primitive value, often used as an identifier for object properties. It ensures that property keys are unique and do not collide with others
- BigInt: A **BigInt** is a numeric data type that can represent integers of arbitrary precision. It was introduced to handle numbers beyond the **Number.MAX_SAFE_INTEGER** ($2^{53} - 1$) limit

40. Describe the concept of "functional programming" in JavaScript. What are some key principles and benefits?

- Functional programming in JavaScript is a style of coding that focuses on using pure functions (no side effects), immutable data (data doesn't change), and declarative code (what to do, not how to do it). It uses features like **map**, **filter**, and **reduce** to process data, avoids modifying external state, and builds reusable, predictable, and clean code.

41. Local Storage, Session Storage, IndexedDB, Cookies ?

- **Local Storage**: Use for simple, long-term key-value storage. Persistent, 4-5MB
- **Session Storage**: Use for temporary data tied to a single tab session. Not persistent storage. 4-5 MB
- **IndexedDB**: Use for complex, large datasets like offline apps. persistent storage. Larger, depending on browser.
- **Cookies**: Use for server communication, authentication, and session tracking. Can be updated by server, short size, 4kb max,

42. What is Hydration ?

- Is a technique to inject javascript code after 1st rendering of an html response from server. It is used in react like library to add interactivity in websites from the client side.

43. What is SSR, SSG, CSR, ISR?

- SSR (Server side rendering): **SSR** (Server-Side Rendering) generates HTML on the server for faster initial page load
- **SSG** (Static Site Generation) pre-builds pages at build time for optimal performance,
- **CSR** (Client-Side Rendering) relies on the browser to render content dynamically,

- **ISR** (Incremental Static Regeneration) allows static pages to be updated after deployment without rebuilding the entire site.

44. What is `.npmrc` when do we use it?

- Is a config file for npm. Used to manage npm install packages. Can define many pre-defined configurations in it.

45. Errors in Javascript?

- **Syntax Error**

```
if (true { // Missing closing parenthesis
  console.log("Hello");
}
```
- **ReferenceError**

```
console.log(x); // ReferenceError: x is not defined
```
- **TypeError**

```
let num = 10;
num.toUpperCase(); // TypeError: num.toUpperCase is
not a function
```
- **RangeError**

```
let arr = new Array(-1); // RangeError: Invalid array length
```

45. Output?

```
function bar() {
  return
  {
    x: 10
  };
}
console.log(bar());
```

46. Output?

```
var x = 10;
(function() {
  console.log(x);
  var x = 20;
})();
```


Next Js Today Task My End :

1. 1) How can you optimize the performance of API routes in Next.js?
2. 2) How would you handle heavy computational tasks in Next.js API routes?
3. 3) What are some strategies for handling long-running API requests in Next.js?
4. 4) Explain how to implement middleware in Next.js API routes to handle reusable logic (e.g., authentication, logging).
5. 5) What are the best practices for securing Next.js API routes?
6. 6) How do you handle API versioning in Next.js?
7. 7) How would you implement rate limiting for Next.js API routes?
8. 8) How would you manage database connections in a serverless environment like Next.js API routes?

5. OOP

1. Do static methods get inherited?

Answer: Yes, the static method gets inherited, but We can't access it through an instance of child class. We have to access it directly using Child class.

```
class A {  
    Static hey() {  
        console.log("Hey");  
    }  
}  
  
class B extends A {};  
  
B.hey()
```

2. Is function overloading available in Javascript OOP?

- No, Javascript doesn't support method overloading. But we can achieve that feature with writing logics, or using default value in params but it doesn't feel like function overloading in other languages.

3. What is method overriding in OOP?

- Method overriding is replacing the Parent class's method with child class.

4. Features of OOP?

- a. **Inheritance**: Having the features from the parent class.
- b. **Polymorphism**: Replacing the methods / variables of parent class with function overriding, overloading.
- c. **Abstraction**: Hiding the implementation of logics/ methods.
- d. **Encapsulation**: Having all the features in a single pack / Object

Asked By Bkash Interview

HTML

1. Describe step by step <script> how it works/loads js while we use it inside <head> tag.
2. How to load a 700kb image in an optimized way.
3. Define <svg> viewBox. Difference between height, width & viewBox numbers.
4. SEO tags ?

CSS

5. There is an image & a description aligned horizontally on the desktop. In mobile i need to align the image to the top & description below. How do you do that?
6. There is a description, an image & another description aligned horizontally on the desktop. In the mobile view I need to align the image to the top & descriptions below. How do you do that?
7. How do you take a button to the bottom right corner?
8. How do you use animation like fade, left to right etc using css?
9. What about Grid?
10. There is a heading followed by a paragraph, I want something. If there is a heading, the paragraph's font size will be increased. (didn't understand the proper questions)
11. difference between rem & em?
12. difference between %, vh,vw,dvh, dvw
13. Margin & padding do work with Inline tags?
14. Block, inline, inline-block ?
15. Animation like fade or left to right move ?

React.js

16. What is HOC?
17. How do you design a Button Component?

Javascripts

1.

```
resolveAll([1, 2, 3, 4]).then(console.log).catch(console.error);  
  
function double(n) {  
  return n * 2;  
}  
  
async function resolveAll(numbers) {  
  return Promise.all(numbers.map(double).map(wait));  
}  
  
function wait(delay) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      if (delay % 2) {  
        reject("E");  
      }  
    }, delay);  
    resolve();  
  });  
}
```

```
    } else {  
        resolve("S");  
    }  
    }, delay);  
});  
}
```

2.

```
const a = ["a", "b", ["c", "d", ["e", ["f"], "g"], "h"], "i", ["j"]]; //output:  
['a','b','c','d','e','f','g','h','i','j']
```

3.

```
[15, 45, 2, 3, 4, 10, 11, 22].sort();
```

4.

```
async function call(a) {  
    return a;  
}  
  
console.log(call(23));
```

5.

```
let a = 10;  
console.log(a)  
let a=20;
```

6.

```
var a = 10;  
console.log(a)  
console.log(b)  
var b=20;
```

7.

```
let a = 10;  
if(a>10) {  
    let a =20;  
}
```

8.

```
const a = {  
    name: "Jhon doe",
```

```

    address: {
      name: "Dhaka"
    }
  };
const b = {...a};
b.address.name = "Khulna";
b.name = "Jahid hasan";

console.log(a.name)
console.log(b.name)
console.log(a.address.name)
console.log(b.address.name)

```

System Design:

1. How do you design the HSC result publishing system?
2. While you sign up into gmail, in the email field, it suggests to you that this email is available or not, how would you implement that? [bloom filter]

```

// const delay = 2000;

// // =====
// asyncFunction().then(console.log); // asyncFunction
// whileLoop("while", console.log); //while
// setTimeout(() => console.log("timeout"), delay); //timeout

// // =====

// async function asyncFunction() {
//   return new Promise((resolve) => {
//     whileLoop("asyncFunction", resolve);
//   });
// }

// function whileLoop(output, fn) {
//   const start = new Date().getTime();
//   while (true) {
//     if (new Date().getTime() - start > delay) {
//       fn(output);
//       break;
//     }
//   }
// }

```

```
// }  
// }  
// }  
  
// async function a () {  
//     // console.log("a")  
//     return 10;  
// }  
  
// console.log(a())
```

1. Difference between merge & rebase
2. In a card component, when you hover on a card, the header of the card component's color will be changed using tailwind.
3. Why do we use keys in lists while rendering using map in react.
4. Controller & uncontrolled component?
5. useLayoutEffects ?
6. Computed value ?
7. What hook gets called first when you use next auth?
8. What are Render props? Write an example of using render props.
9. LocalStorage & session storage diff?
- 10.